

CSE4067 Introduction to Blockchain Programming: Project

goshgar.ismayilov@boun.edu.tr

November 2024

Part I: Proposal

Find a sufficiently-challenging real-world problem that you can solve or improve with the help of decentralized application (i.e. **dApp**) over blockchain. Reading *Do you need a blockchain?* paper (shared in Google Classroom) before crafting your problem is recommended.

Each application must have at least the following system components. The inclusion of any further components is also welcome.

- inclusion of at least one custom **ERC-20** or **ERC-721** token contract
- fully-functional service contract(s) deployed to testnet
- fully-functional web interface(s) interacting with smart contract(s)

Prepare **1-page** brief project proposal, explaining project details (e.g. motivation, system components and challenges to overcome). All proposals need instructor approval. In case of disapproval, the proposal must be urgently fixed and resubmitted for approval.

- Due date: Within **1-week** after announcement

Part II: Implementation

Solidity must be used to implement smart contract(s).

The testnet for contract deployment is upon to your decision.

- **Sepolia** is recommended.

Web interface must connect to **Metamask** and interact with smart contract(s).

- The development platform for the interface is upon to your decision.
- **JavaScript** is recommended to be able to use **web3.js** or **ethers.js**.

Implementation must be well-commented with proper styling. Quality of comments will be also evaluated.

Part III: Documentation & Demo

Prepare comprehensive report to explain system design and architecture including individual components (e.g. smart contract(s), web interface(s), etc.) and interaction between these components. Measure and provide performance of your system (e.g. blockchain gas consumption, interface response times, throughput, latency, etc.). Support your findings with proper figures and tables. Cover the challenges you have overcome during the implementation phase and the points you can improve more about the existing system.

The report must contain the address(es) of the smart contract(s) deployed on the testnet. Ensure that the contract(s) is successfully deployed before due date and always externally accessible.

Timestamp your entire project before you submit it. Keep the project files and the resulting `.ots` file(s).

- <https://opentimestamps.org/>

Prepare for a 10-minute in-class live-demo presentation.

This project must be done in groups of 2 or 3 students, with 7-week long at total after its announcement. Cheating (e.g. high similarity between assignments, help from AI tools) will be penalized. Submit your project with the following format. Single submission from the group is sufficient.

- 2 Students: `#No1-#No2.zip`
- 3 Students: `#No1-#No2-#No3.zip`